📘 Backend

📂 Estrutura de Pastas do Backend (Spring Boot)

Nosso projeto segue uma arquitetura em camadas, comum em aplicações Spring Boot, que separa responsabilidades e facilita a manutenção, os testes e a escalabilidade.

1️⃣ Model

- Local onde ficam as **entidades** do sistema.
- Cada classe dentro de `model` representa uma **tabela no banco de dados** ou um **objeto de negócio**.
- Exemplo: `Sala`, `Professor`, `Aluno`.

BYTECRAFT

2️⃣ Repository

- Contém as **interfaces de acesso ao banco de dados**.
- Geralmente estendem `JpaRepository` ou `CrudRepository`.
- São responsáveis por operações como **salvar, buscar, atualizar e deletar** registros.
- Exemplo: `SalaRepository`, `ProfessorRepository`.
- Anotação principal: `@Repository` (opcional, pois o Spring já detecta ao estender JpaRepository).

3️⃣ Service

- Camada que contém a **lógica de negócio** do sistema.
- Aqui ficam as **regras e validações** antes de acessar o banco.
- Evita que a lógica fique diretamente no `controller`.
- Exemplo: `SalaService`, `ProfessorService`.
- Anotação principal: `@Service`.

4️⃣ Controller

- Responsável por expor os **endpoints da API** (REST).
- É a camada que se comunica com o **frontend ou clientes externos**.
- Recebe requisições HTTP (`GET`, `POST`, `PUT`, `DELETE`) e retorna respostas (`ResponseEntity`, `DTOs`).
- Exemplo: `SalaController`, `ProfessorController`.
- Anotação principal: `@RestController`.

BYTECRAFT

5️⃣ DTO (Data Transfer Object)

- Objetos usados para **transferir dados entre camadas**.
- Servem para **não expor diretamente as entidades do banco**.
- Podem representar:
    - Dados de entrada (ex.: `CreateSalaDTO`).
    - Dados de saída (ex.: `SalaResponseDTO`).
- Vantagens:
    - Maior segurança.

- Flexibilidade para mudar a entidade sem quebrar a API.

6️⃣ Enums

- Pastas que contêm **tipos enumerados** usados no sistema.
- Exemplo: `StatusAluno { ATIVO, INATIVO, SUSPENSO }`.
- Usados para valores fixos, evitando erros e facilitando validações.

7️⃣ Security

- Pasta responsável pela **segurança da aplicação**.
- Contém classes para:
  - Configuração de autenticação e autorização.
  - Filtros de segurança (JWT, OAuth2, etc.).
  - Implementação do `UserDetailsService` e `PasswordEncoder`.
- Exemplo: `SecurityConfig`, `JwtFilter`, `UserDetailsImpl`.

📘 Entidade `Aluno`

A entidade **Aluno** é um recurso central do sistema.
Seu fluxo segue a arquitetura em camadas do Spring Boot:

- **Model (`Aluno`)** → representação da tabela no banco.
- **Repository (`AlunoRepository`)** → acesso ao banco.
- **Service (`AlunoService`)** → regras de negócio.

- **Controller (`AlunoController`)** → endpoints da API.
- **DTO (`AlunoDTO`)** → transferência de dados para o frontend.

1️⃣ Model: `Aluno.java`

- Representa a tabela **`alunos`**.
- Principais atributos:
  - `id`: chave primária técnica.
  - `apelido`: identificador único dentro de cada sala.
  - `nivel`: enum (`FACIL`, `MEDIO`, `DIFICIL`).
  - `sala`: relacionamento muitos-para-um (`ManyToOne`) com `Sala`.
- Restrições:
  - **apelido + sala_id** → devem ser únicos.
- Anotações importantes:
  - `@Entity`, `@Table`, `@Enumerated`, `@ManyToOne`.

BYTECRAFT

2️⃣ Repository: `AlunoRepository.java`

Interface que estende `JpaRepository<Aluno, Long>`.
Permite **CRUD automático** e define consultas customizadas:

```
Optional<Aluno> findByApelidoAndSala(String apelido, Sala sala);
```

🔹 Busca aluno pelo apelido dentro de uma sala.

```
Optional<Aluno> findByApelido(String apelido);
```

- Busca aluno apenas pelo apelido.

```
@Query("UPDATE Aluno a SET a.nivel = :nivel WHERE a.apelido = :apelido
AND a.sala = :sala")

int atualizaNivel(NivelDificuldadeEnum nivel, String apelido, Sala
sala);
```

- Atualiza o nível de dificuldade de um aluno, considerando apelido + sala.

---

3 Service: `AlunoService.java`

Contém as regras de negócio para manipular alunos.

```
public Aluno vincularAlunoASala(String apelido, Byte codigoSala)
```

- Busca a sala pelo código único.
- Se aluno já existe na sala, retorna o existente.
- Caso contrário, cria um novo aluno e salva no banco.

```
public void registraNivel(Aluno aluno)
```

- Atualiza o nível do aluno via query customizada no repositório.
- Marcado como `@Transactional` para garantir consistência.

```
public Aluno findAluno(String apelido, Byte codigoSala)
```

- Busca uma sala pelo código.
- Retorna o aluno correspondente ou lança erro se não encontrado.

---

4 DTO: `AlunoDTO.java`

Objeto de transferência de dados.

Campos:

- `apelido` → apelido do aluno.
- `nivel` → nível de dificuldade em formato `String`.
- `sala` → objeto `SalaDTO` com dados da sala associada.

```
public static AlunoDTO fromEntity(Aluno aluno)
```

- Converte a entidade `Aluno` para `AlunoDTO`.
- Inclui informações da sala (`SalaDTO`).

📌 Usado para enviar respostas mais seguras e organizadas ao frontend.

5️⃣ Controller: `AlunoController.java`

Exposição dos **endpoints REST** para o recurso `Aluno`.

Base URL: `/api/alunos`

🔹 `POST /login`

- Faz o **login ou registro automático** de um aluno em uma sala.
- Regras:
  - `apelido` obrigatório.
  - `codigoSala` obrigatório.

Resposta JSON:

```
{
  "apelido": "João",
  "nivel": "FACIL",
```

```
  "sala": {
    "id": 1,
    "nomeTurma": "Turma A",
    "codigo": 10
  }
}
```

-

🔹 GET /niveis

Retorna a lista de níveis de dificuldade disponíveis:

```
["FACIL", "MEDIO", "DIFICIL"]
```

-

🔹 POST /{apelido}/registrarNivel

- Atualiza o nível de dificuldade de um aluno.
- Regras:
    - Precisa do `codigoSala`.
    - Precisa do `nivel` em formato string (`FACIL`, `MEDIO`, `DIFICIL`).
- Retorna `AlunoDTO` atualizado.

📘 Entidade Sala

A entidade **Sala** representa uma turma no sistema.
Seu fluxo também segue a arquitetura em camadas do Spring Boot:

Model (Sala) → representação da tabela no banco.
Repository (SalaRepository) → acesso ao banco.
Service (SalaService) → regras de negócio.
Controller (SalaController) → endpoints da API.
DTO (SalaDTO) → transferência de dados para o frontend.

1️⃣ Model: `Sala.java`

Representa a tabela **salas**.

Principais atributos:

- `id` → chave primária técnica (auto-incremento).
- `nomeTurma` → nome identificador da turma (ex.: "Turma A").
- `codigoUnico` → código numérico entre 10–99, usado para vinculação de alunos.
- `alunos` → relacionamento **um-para-muitos** (OneToMany) com a entidade `Aluno`.

Restrições:

- `codigoUnico` deve ser **único** no banco.
- `nomeTurma` não pode ser nulo.

Anotações importantes:

- `@Entity`, `@Table`
- `@OneToMany(mappedBy = "sala")`
- `@JsonManagedReference` → evita loop de serialização com `Aluno`.

2️⃣ Repository: `SalaRepository.java`

Interface que estende `JpaRepository<Sala, Long>`.
Permite CRUD automático e consultas customizadas:

```
Optional<Sala> findByCodigoUnico(Byte codigoUnico);
```

- Busca sala pelo código único.

```
Optional<Sala> findByNomeTurma(String nomeTurma);
```

- Busca sala pelo nome.

```
List<Sala> findAll();
```

- Retorna todas as salas cadastradas.



3️⃣ Service: `SalaService.java`

Contém as regras de negócio relacionadas às salas.

```
private Byte geraCodigoUnico()
```

- Gera um número aleatório entre 10 e 99, usado como código único da sala.

```
public Sala criaSala(String nomeTurma)
```

- Se já existir uma sala com o mesmo nome, retorna a existente.
- Caso contrário, cria uma nova sala com código único gerado automaticamente.

```
public Optional<Sala> getSalaByCodigo(Byte codigo)
```

- Busca sala pelo código único.

```
public List<Sala> getTodasSalas()
```

- 🔹 Retorna todas as salas registradas no banco.

4️⃣ DTO: `SalaDTO.java`

Objeto de transferência de dados da Sala.

Campos:

- `id` → identificador técnico da sala.
- `nomeTurma` → nome da turma.
- `codigoUnico` → código usado para vinculação de alunos.

```
public static SalaDTO fromEntity(Sala sala)
```

- 🔹 Converte entidade Sala para DTO.

BYTECRAFT

5️⃣ Controller: `SalaController.java`

Exposição dos endpoints REST para o recurso Sala.
Base URL: `/api/salas`

🔹 POST `/criar`

Cria uma nova sala no sistema.

Regras:

- `nomeTurma` obrigatório.

Resposta JSON:

```
{
  "id": 1,
  "nomeTurma": "Turma A",
```

```
  "codigoUnico": 42
}
```

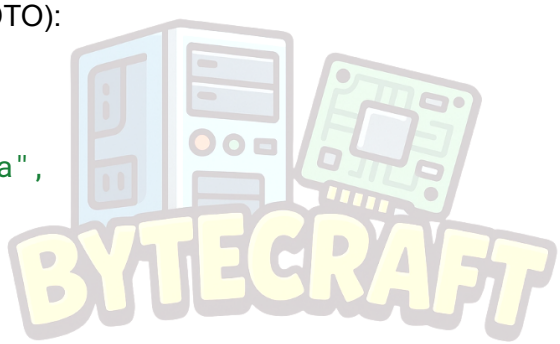
🔹 POST `/vincular`

Vincula um aluno a uma sala existente.

Regras:

- `apelido` obrigatório.
- `codigoSala` obrigatório.

Resposta JSON (AlunoDTO):




```
{
  "apelido": "Maria",
  "nivel": null,
  "sala": {
    "id": 1,
    "nomeTurma": "Turma A",
    "codigoUnico": 42
  }
}
```

🔹 GET `/existe`

Verifica se existem salas cadastradas.

Resposta JSON:

```
{
  "existe": true,
  "salas": [
    {
      "id": 1,
      "nomeTurma": "Turma A",
      "codigoUnico": 42
    }
  ]
}
```

📘 Entidade Professor

A entidade **Professor** representa os usuários que administram as turmas no sistema.
Ela segue a arquitetura em camadas do Spring Boot:

Model (Professor) → representação da tabela no banco.
Repository (ProfessorRepository) → acesso ao banco (via `EntityManager`).
Service (ProfessorService) → regras de negócio (cadastro, autenticação).
Controller (ProfessorController) → endpoints da API.
DTO (ProfessorDTO) → transferência de dados para o frontend.

1️⃣ Model: `Professor.java`

Representa a tabela **professores**.

Principais atributos:

- `id` → chave primária técnica (auto-incremento).
- `nomeDeUsuario` → identificador único do professor.

- `senha` → armazenada de forma codificada (hash).
- `sala` → relacionamento **muitos-para-um** com `Sala` (um professor pertence a uma sala).

Restrições e anotações:

- `@Entity`, `@Table`
- `@Column(nullable = false)` → `nomeDeUsuario` e `senha` obrigatórios.
- `@ManyToOne` → relacionamento com `Sala`.

2️⃣ Repository: `ProfessorRepository.java`

Classe que utiliza o **EntityManager** para persistência manual (ao invés de `JpaRepository`).

Métodos principais:

```
public int salvaProfessor(Professor professor)
```

🔹 Persiste o professor no banco.
Retorna `1` em caso de sucesso ou `0` em caso de falha.

```
public Professor buscarPorNome(String nomeDeUsuario)
```

🔹 Busca um professor pelo nome de usuário.
Retorna `null` caso não seja encontrado.

3️⃣ Service: `ProfessorService.java`

Contém as regras de negócio para professores.

```
public static boolean cadastrarProfessor(String nome, String
senha, String nomeTurma,
ProfessorRepository professorRepo,SalaService salaService,
PasswordEncoder passwordEncoder)
```

- 🔹 RN02 → Senha deve ter no mínimo **6 caracteres**.
- 🔹 RN03 → Nome de usuário deve ser **único**.
- 🔹 Cria ou obtém a sala informada.
- 🔹 Codifica a senha com `PasswordEncoder`.
- 🔹 Persiste professor no banco.

```
public static boolean autenticarProfessor(String nome, String
senha, ProfessorRepository professorRepo,
PasswordEncoder passwordEncoder)
```

- 🔹 Busca professor pelo nome.
- 🔹 Compara senha informada com hash armazenado (`passwordEncoder.matches`).
- 🔹 Retorna `true` se autenticado, senão `false`.

4️⃣ DTO: `ProfessorDTO.java`

Objeto de transferência de dados para o frontend.

Campos:

- `nomeDeUsuario` → identificador do professor.
- `sala` → objeto `SalaDTO` associado.

```
public static ProfessorDTO fromEntity(Professor professor)
```

- 🔹 Converte entidade Professor em `ProfessorDTO`.

📌 Evita expor diretamente a senha ou outros dados sensíveis.

5️⃣ Controller: `ProfessorController.java`

Exposição dos endpoints REST do recurso Professor.
Base URL: `/api/professores`

🔹 POST `/cadastrar`

Cadastra um professor no sistema e cria automaticamente uma sala caso não exista.

Regras:

- Nome de usuário único.
- Senha mínima de **6 caracteres**.
- Nome da turma obrigatório.

BYTECRAFT

Resposta JSON (sucesso):

```
{
  "nomeDeUsuario": "professor01",
  "nomeTurma": "Turma A",
  "sala": {
    "id": 1,
    "nomeTurma": "Turma A",
    "codigo": 42
  }
}
```

Resposta JSON (erro):

```
{ "erro": "Nome de usuário já existe" }
```

🔹 POST `/autenticar`

Autentica o professor no sistema.

Regras:

- `nome` e `senha` obrigatórios.
- Senha validada contra o hash armazenado.

Resposta JSON (sucesso):

```
{
  "nomeDeUsuario": "professor01",
  "nomeTurma": "Turma A",
  "sala": {
    "id": 1,
    "nomeTurma": "Turma A",
    "codigo": 42
  }
}
```

BYTECRAFT

Resposta JSON (falha):

```
{ "erro": "Nome de usuário ou senha inválidos." }
```

📘 SecurityConfig

A classe **SecurityConfig** configura a **segurança da aplicação** usando Spring Security.
Ela define como as requisições HTTP são autorizadas, habilita CORS e fornece codificação de senhas.

1️⃣ Anotações da classe

```
@Configuration
```

- Marca a classe como **bean de configuração do Spring**, carregada no contexto da aplicação.

2️⃣ SecurityFilterChain

```
@Bean

    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception
```

Função:

- Configura regras de **autorização e autenticação**.

BYTECRAFT

Configurações importantes:

- `.csrf(csrf -> csrf.disable())`
  - Desativa CSRF apenas para testes (útil quando frontend está separado).
- `.cors(cors -> cors.configurationSource(corsConfigurationSource()))`
  - Habilita **CORS** e define a fonte de configuração.
- `.authorizeHttpRequests(auth -> auth ... )`
  - Define permissões para endpoints:
    - `/api/alunos/**` → liberado para todos.
    - `/api/professores/**` → liberado para todos (login/cadastro).
    - `/api/salas/**` → liberado para todos.

- `anyRequest().authenticated()` → qualquer outro endpoint exige autenticação.

3️⃣ CorsConfigurationSource

```
@Bean

    public CorsConfigurationSource corsConfigurationSource()
```

Função:

- Configura **Cross-Origin Resource Sharing (CORS)**, permitindo que o frontend acesse a API sem bloqueios.

Configurações:

- `setAllowedOrigins(List.of("http://localhost:3030"))` → libera apenas o frontend React.
- `setAllowedMethods(List.of("GET","POST","PUT","DELETE","OPTIONS"))` → métodos HTTP permitidos.
- `setAllowedHeaders(List.of("*"))` → permite todos os headers.
- `setAllowCredentials(true)` → necessário para enviar cookies e autenticação em POST.

4️⃣ PasswordEncoder

```
@Bean

    public PasswordEncoder passwordEncoder() {

        return new BCryptPasswordEncoder();

    }
```

Função:

- Fornece o **bean de codificação de senha** usado pelo `ProfessorService`.
- `BCryptPasswordEncoder` → algoritmo seguro para armazenamento de senhas.
- Usado em:
    - Cadastro: para **hash da senha** antes de salvar no banco.
    - Login: para **comparar senha informada com hash** armazenado.

📘 Dependências do Projeto

O projeto **Bytecraft** utiliza Maven para gerenciar bibliotecas e recursos externos.

1️⃣ Spring Boot Starters

Facilitam a configuração e inclusão de recursos comuns do Spring Boot:

- **spring-boot-starter-data-jpa**
  🔹 Suporte a **JPA/Hibernate** para persistência de dados em bancos relacionais.
- **spring-boot-starter-jdbc**
  🔹 Suporte a **JDBC** para acesso direto a banco de dados, quando necessário.
- **spring-boot-starter-security**
  🔹 Integração com **Spring Security** para autenticação e autorização.
- **spring-boot-starter-web**
  🔹 Criação de **APIs REST**, suporte a Spring MVC, JSON e servidores embutidos (Tomcat).

2️⃣ Lombok

```
<dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <version>1.18.32</version>
            <scope>provided</scope>
  </dependency>
```

- Reduz **código boilerplate** (getters, setters, construtores, toString).
- `scope=provided` → não incluído no JAR final, apenas em compilação.

3️⃣ MySQL

```
<dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
</dependency>
```

- Conector JDBC para **banco de dados MySQL**.
- `scope=runtime` → necessário apenas em tempo de execução.

4️⃣ DevTools

```
<dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
```

BYTECRAFT

- Ferramenta de **desenvolvimento rápido**.
- Habilita **reload automático** ao alterar código ou templates.

5️⃣ Testes

- **spring-boot-starter-test** → suporte a **testes unitários e de integração** (JUnit, Mockito, AssertJ).
- **spring-security-test** → suporte a **testes de segurança** (Spring Security, autenticação, autorização).

📘 API Frontend – Integração com o Backend

O arquivo `api.ts` define funções para **comunicação com o backend Spring Boot** via **HTTP (fetch API)**.
Ele mantém a tipagem usando **TypeScript**, garantindo consistência nos dados.

1️⃣ Configuração Base

```
const API_BASE_URL = 'http://localhost:8080/api'; // endereço completo do backend
```

- URL base para todos os endpoints do backend.
- Facilita mudanças de ambiente (desenvolvimento, produção).

2️⃣ Interfaces de Tipagem

```
export interface ApiAluno {
  apelido: string;
  nivel?: string;
  turma?: string;
}

export interface ApiSala {
  id: number;
  nomeTurma: string;
  codigoUnico: number;
}

export interface ApiProfessor {
  nomeDeUsuario: string;
```

```
  senha?: string;

  nomeTurma?: string;

  sala?: ApiSala;

}
```

- Define **formato esperado dos objetos** retornados pelo backend.
- Garante que o frontend saiba quais campos estarão disponíveis.

3️⃣ Funções – Aluno

loginAluno

```
export const loginAluno = async (apelido: string, codigoSala: string)
```

- Faz **login ou registro automático** do aluno.
- Chama endpoint: **POST /api/alunos/login**
- Retorna objeto `ApiAluno`.
- Valida erros do backend e lança exceção em caso de falha.

getNiveis

```
export const getNiveis = async ()
```

- Busca lista de **níveis de dificuldade disponíveis** (`FACIL`, `MEDIO`, `DIFICIL`).
- Chama endpoint: **GET /api/alunos/niveis**
- Retorna array de strings em **lowercase**.

registrarNivel

```
export const registrarNivel = async (apelido: string, nivel: string, codigoSala: number)
```

- Atualiza o **nível de dificuldade do aluno**.
- Chama endpoint: **POST /api/alunos/{apelido}/registrarNivel**
- Retorna o aluno atualizado (`ApiAluno`).

vincularAlunoASala

```
const vincularAlunoASala = async (apelido: string, codigoSala: string)
```

- Vincula um aluno a uma sala existente.
- Chama endpoint: **POST /api/salas/vincular**
- Retorna `ApiAluno` com informações de apelido, nível e turma.

4️⃣ Funções – Sala

cadastrarSala

```
export const cadastrarSala = async (nomeTurma: string)
```

- Cria uma nova sala no backend.
- Chama endpoint: **POST /api/salas/criar**
- Retorna objeto `ApiSala` com `id`, `nomeTurma` e `codigo`.

listarSalas

```
export const listarSalas = async ()
```

- Retorna lista de todas as salas cadastradas.
- Chama endpoint: **GET /api/salas/listar**
- Retorna array de objetos `ApiSala`.

5️⃣ Funções – Professor

cadastrarProfessor

```
export const cadastrarProfessor = async (nomeDeUsuario: string,senha: string,nomeTurma: string)
```

- Cria um professor e vincula a sala correspondente.
- Chama endpoint: **POST /api/professores/cadastrar**
- Retorna `ApiProfessor` com dados da sala criada.

loginProfessor

```
export const loginProfessor = async (nomeDeUsuario: string, senha: string)
```

- Autentica o professor no backend.
- Chama endpoint: **POST /api/professores/autenticar**
- Retorna `ApiProfessor` com dados da sala vinculada.

BYTECRAFT

6️⃣ Objeto Consolidado

```
export const api = {

  // Alunos

  loginAluno,

  getNiveis,

  registrarNivel,

  vincularAlunoASala,



  // Salas
```

```
  listarSalas,

  cadastrarSala,



  // Professores

  cadastrarProfessor,

  loginProfessor,

};
```

- Permite **importar todas as funções de API** em um único objeto.
- Facilita chamadas e manutenção do frontend.

🚀 Rodando o Backend (Spring Boot)

Antes de iniciar, é necessário garantir que o **banco de dados MySQL** esteja criado e configurado corretamente.

1️⃣ Criar o Banco de Dados (se não existir)

Se você **não possui o banco `bytecraft`**, siga os passos abaixo:

1. Acesse o MySQL:

```
#Se o MySQL pedir senha para o root:

     mysql -u root -p

#Se o root do MySQL estiver vinculado ao root do sistema (com sudo):

     sudo mysql -u root
```

2. Crie o banco e o usuário:

```
CREATE DATABASE bytecraft CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;

CREATE USER 'bytecraft_user'@'localhost' IDENTIFIED BY 'senha123';

GRANT ALL PRIVILEGES ON bytecraft.* TO 'bytecraft_user'@'localhost';

FLUSH PRIVILEGES;
```

3. Teste o login com o novo usuário:

```
mysql -u bytecraft_user -p

Digite a senha (senha123) quando solicitado.


SHOW DATABASES;
```

✅ Você deverá ver o banco `bytecraft`.

2️⃣ Ajustes caso o banco já exista

Se você **já possui o banco `bytecraft`**, **atenção**:

- A coluna `codigo` da tabela `salas` **não é mais usada**.
- Para evitar conflitos, execute:

```
ALTER TABLE salas DROP COLUMN codigo;
```

> Isso garante que a nova lógica de geração de código único (`codigoUnico`) funcione corretamente.

3️⃣ Configurar o application.properties

O projeto já vem com configuração padrão:

```
spring.application.name=bytecraft

spring.datasource.url=jdbc:mysql://localhost:3306/bytecraft?useSSL=fals
e&serverTimezone=UTC

spring.datasource.username=bytecraft_user

spring.datasource.password=senha123

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
```

Se você mudou usuário ou senha, atualize aqui.

4️⃣ Compilar e Rodar o Backend

No terminal, dentro da pasta do backend (onde está o `pom.xml`):

```
### Compilar o projeto

mvn clean install

### Rodar a aplicação

mvn spring-boot:run
```

- A aplicação ficará disponível em: `http://localhost:8080/api`
- Endpoints principais:
  - `/api/alunos/**` → CRUD de alunos e registro de níveis
  - `/api/salas/**` → CRUD de salas e vinculação de alunos
  - `/api/professores/**` → Cadastro e autenticação de professores